

# OPERIVAOS

## Enterprise Operations & Process Platform

---

A unified system for project lifecycle management, multi-level approvals, and cross-department operations — from first client conversation to final delivery, with role-based visibility, full audit history, and an embedded AI assistant.

**Document type:** Complete System & Development Documentation

**Applicable domains:** Corporates · Government bodies · Law firms · NGOs · SMBs

**Architecture style:** Modular monolith, phased build, containerized deployment

Version 1.0 · Prepared as a build & reference document

# Table of Contents

|    |                                                |    |
|----|------------------------------------------------|----|
| 01 | Introduction                                   | 03 |
| 02 | Product Description                            | 04 |
| 03 | Objectives & Key Outcomes                      | 05 |
| 04 | Pros, Cons & Engineering Challenges            | 06 |
| 05 | Full End-to-End Workflow Diagram (Parts A & B) | 07 |
| 06 | High-Level System Architecture                 | 09 |
| 07 | Entity-Relationship Diagram                    | 10 |
| 08 | State Machine Designs                          | 11 |
| 09 | Key Flow Sequence Diagram & RBAC Model         | 12 |
| 10 | AI Assistant & Chatbot Architecture            | 13 |
| 11 | AI / RAG Component Choices                     | 14 |
| 12 | Technology Stack (Full)                        | 15 |
| 13 | Frontend Design System                         | 16 |
| 14 | Docker & Containerization Architecture         | 17 |
| 15 | CI/CD Pipeline                                 | 18 |
| 16 | Deployment Architecture                        | 19 |
| 17 | Development Phases — Overview                  | 20 |
| 18 | Phase 0 & Phase 1 — Setup and Core Lifecycle   | 21 |
| 19 | Phase 2 & Phase 3 — RBAC and Workflow Engine   | 22 |
| 20 | Phase 4 & Phase 5 — Events and AI Assistant    | 23 |
| 21 | Phase 6 — Frontend Build Track                 | 24 |
| 22 | Phase 7 & Phase 8 — Deployment and Launch      | 25 |
| 23 | Full Timeline (Gantt View)                     | 26 |
| 24 | Risks & Mitigations                            | 27 |
| 25 | Closing Summary                                | 28 |

# 1. Introduction

---

## What OPERIVA OS Is

- A single platform that runs a project or engagement from first client conversation through requirement capture, budgeting, multi-level approval, execution, and closure.
- Built to replace scattered email chains, verbal approvals, and disconnected spreadsheets with one auditable digital record.
- Domain-agnostic at the core — the same engine that runs a corporate project pipeline can run a law firm's matter-approval process or a government permit workflow.
- Every stage, decision, and change is logged — nothing lives only in someone's inbox.

## Who It Is For

- Corporates running client or internal project pipelines with multiple approval levels.
- Law firms managing matter intake, client approvals, and billing sign-off.
- Government departments handling permit, tender, or budget-sanction workflows.
- NGOs and SMBs that need structured approvals without enterprise-scale overhead.

## What This Document Covers

- Product description, objectives, and honest trade-offs (pros / cons / challenges).
- Complete workflow diagram — every actor, every layer, every fallback path, in one picture.
- System design: architecture layers, database schema (ER diagram), state machines, sequence flow, RBAC model.
- The AI assistant / chatbot architecture — which AI, how it retrieves data safely, and free vs. paid options.
- Full technology stack with current stable versions.
- Frontend design system, already implemented as a working Angular build.
- Docker, containerization, and CI/CD pipeline design.
- A phase-by-phase development roadmap with week-by-week time estimates.

## 2. Product Description

---

- **Entry point:** a client or internal stakeholder raises a request; the responsible team (e.g. Marketing/BD) opens a new project container on the platform with an auto-generated code — no manual numbering, no spreadsheet.
- **Feasibility check:** the request routes to the CEO/decision layer for a quick go/no-go; on approval, a confirmation is auto-generated and sent to the client — no manually drafted email.
- **Progress history:** every meeting held around the project is logged under a Progress section tied to that project — chronological, searchable, and permanent.
- **Requirements & budget:** captured as structured, **versioned** records, not static documents. Any later change (more devices, increased scope) is appended to the history, never overwritten — a running ledger, not a mutable field.
- **Execution:** once approved, a Manager workspace opens; the Manager assigns tasks to Employees, each of whom sees only the requirements relevant to their own task.
- **Approvals:** proposals and deliverables move up a configurable multi-level chain (Manager → Finance → CEO/Board) with three outcomes at each level — approve, reject, or request clarification.
- **Confidentiality by design:** access is strictly role- and project-scoped. A Manager sees only their assigned projects; an Employee sees only their assigned tasks — never budgets, other projects, or board-level notes.
- **Cross-department requests:** budget, asset, and HR needs are raised as structured tickets that route automatically to the right department — no "who do I even email" problem.
- **AI assistant:** a role-aware chat assistant answers status questions instantly, scoped to exactly what the asker is permitted to see.

### 3. Objectives & Key Outcomes

---

- Eliminate email- and paperwork-driven approval chains in favor of one auditable digital workflow.
- Preserve complete historical context — meetings, requirement changes, budget changes — without manual record-keeping.
- Enforce strict, project-scoped confidentiality so every role sees only what it is entitled to see.
- Cut the time from "request raised" to "decision made" through structured routing and SLA-based auto-escalation.
- Provide instant, permission-respecting status answers via an embedded AI assistant.
- Build the core as a reusable workflow engine — usable across industries, not tied to one domain's vocabulary.
- Ship in clearly bounded phases so a working system exists early, with functionality layered on incrementally.

## 4. Pros, Cons & Engineering Challenges

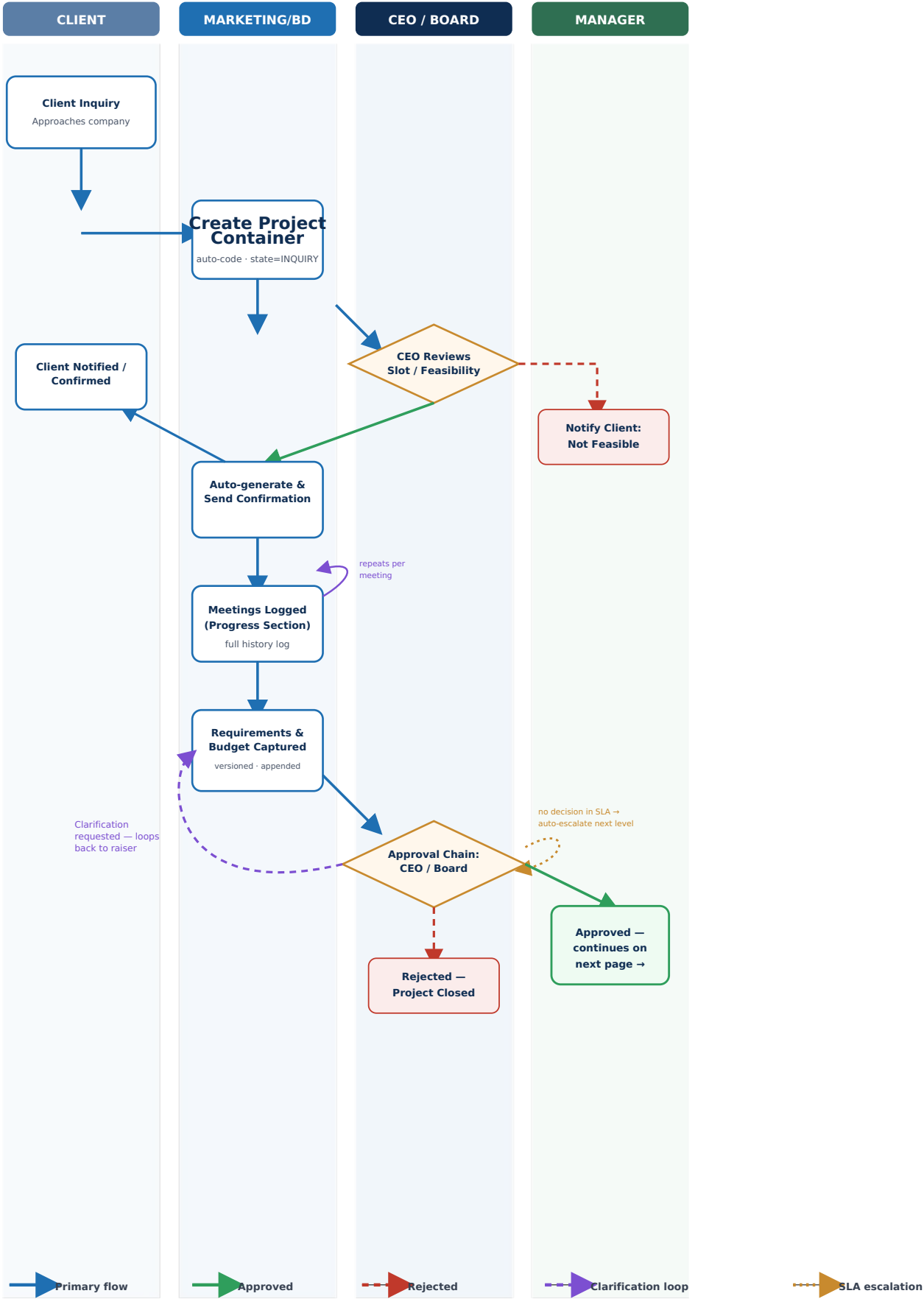
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pros</b> <ul style="list-style-type: none"><li>• Single source of truth for the entire project lifecycle</li><li>• Full, append-only audit trail — strong fit for regulated sectors</li><li>• One generic workflow engine reused across approval types</li><li>• Project-scoped RBAC prevents data leakage by design</li><li>• AI assistant cuts interruption-driven status checks</li><li>• SLA auto-escalation removes the "silently ignored" failure mode</li><li>• Domain-agnostic — usable well beyond one industry</li></ul> | <b>Cons</b> <ul style="list-style-type: none"><li>• Large surface area — risk of over-scoping if built all at once</li><li>• Requires real organizational adoption to replace email habits</li><li>• Fine-grained (project + role) access control is harder than simple RBAC</li><li>• Early data-modeling mistakes are expensive to retrofit later</li><li>• An AI assistant with data access adds a new risk class if scoping is wrong</li></ul> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Key Engineering Challenges

| Challenge                | Why It's Hard                                                                             | Approach                                                                                                                      |
|--------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Project-scoped RBAC      | A role isn't global — "Manager" means different access on different projects              | Model access as (user, project, role) triples, resolved per request, cached in Redis.                                         |
| History without bloat    | Budget/requirement changes must be traceable, but "current" must stay fast to query       | Append-only revision tables with a materialized "current" pointer, never in-place overwrites.                                 |
| Generic workflow engine  | Budget approval, asset requests, and clarification loops look similar but differ in steps | Model an ApprovalChain as an ordered list of ApprovalSteps tied to a request type — new types become configuration, not code. |
| AI assistant scope leaks | An LLM answers with whatever context it's given, regardless of who's asking               | Filter the vector search by the caller's permission scope before retrieval — never rely on prompt instructions alone.         |

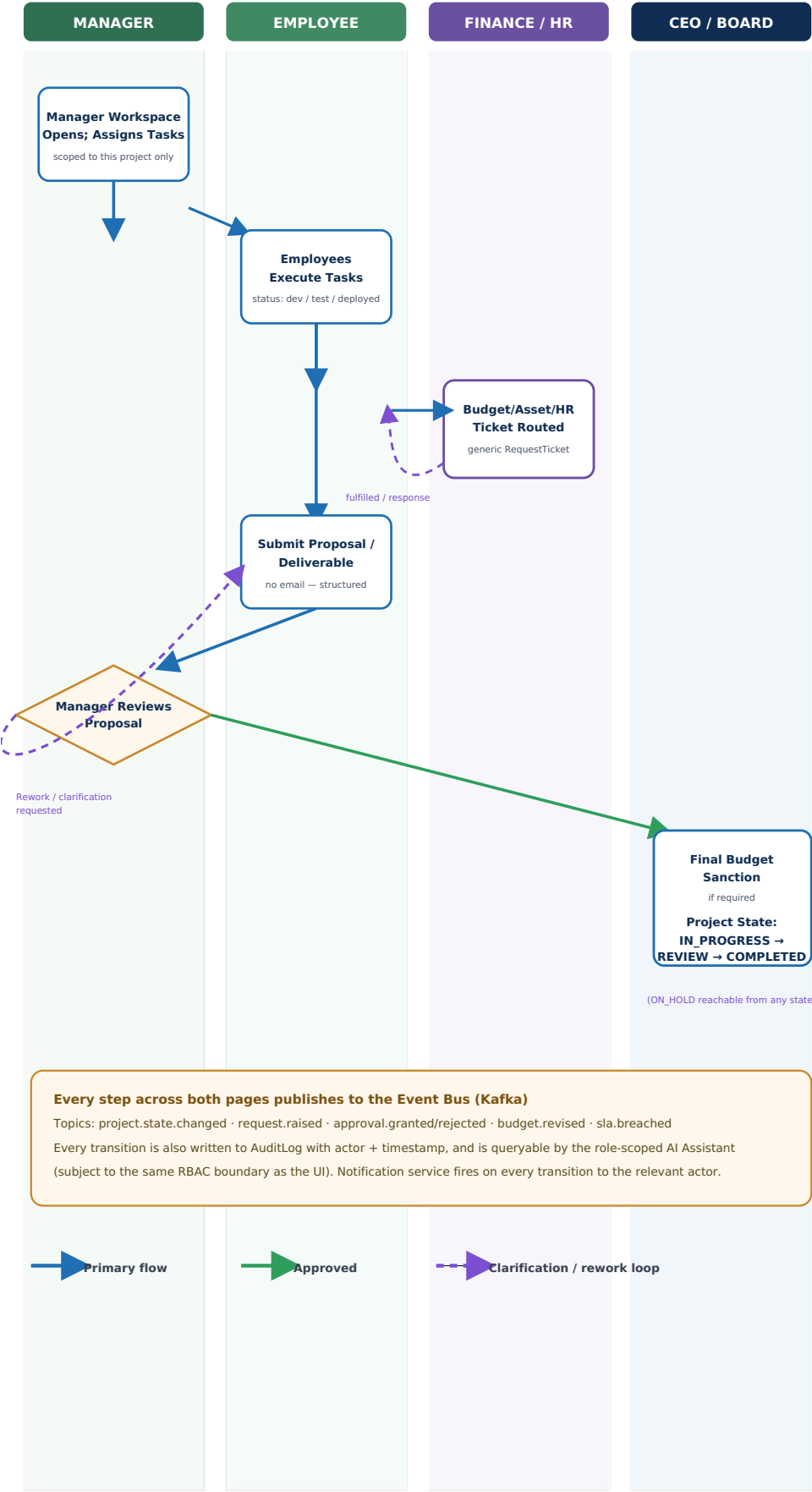
## 5. Full End-to-End Workflow — Part A: Intake to Approval

Actor lanes run left to right; the flow runs top to bottom. Continues on the next page from the approval decision.



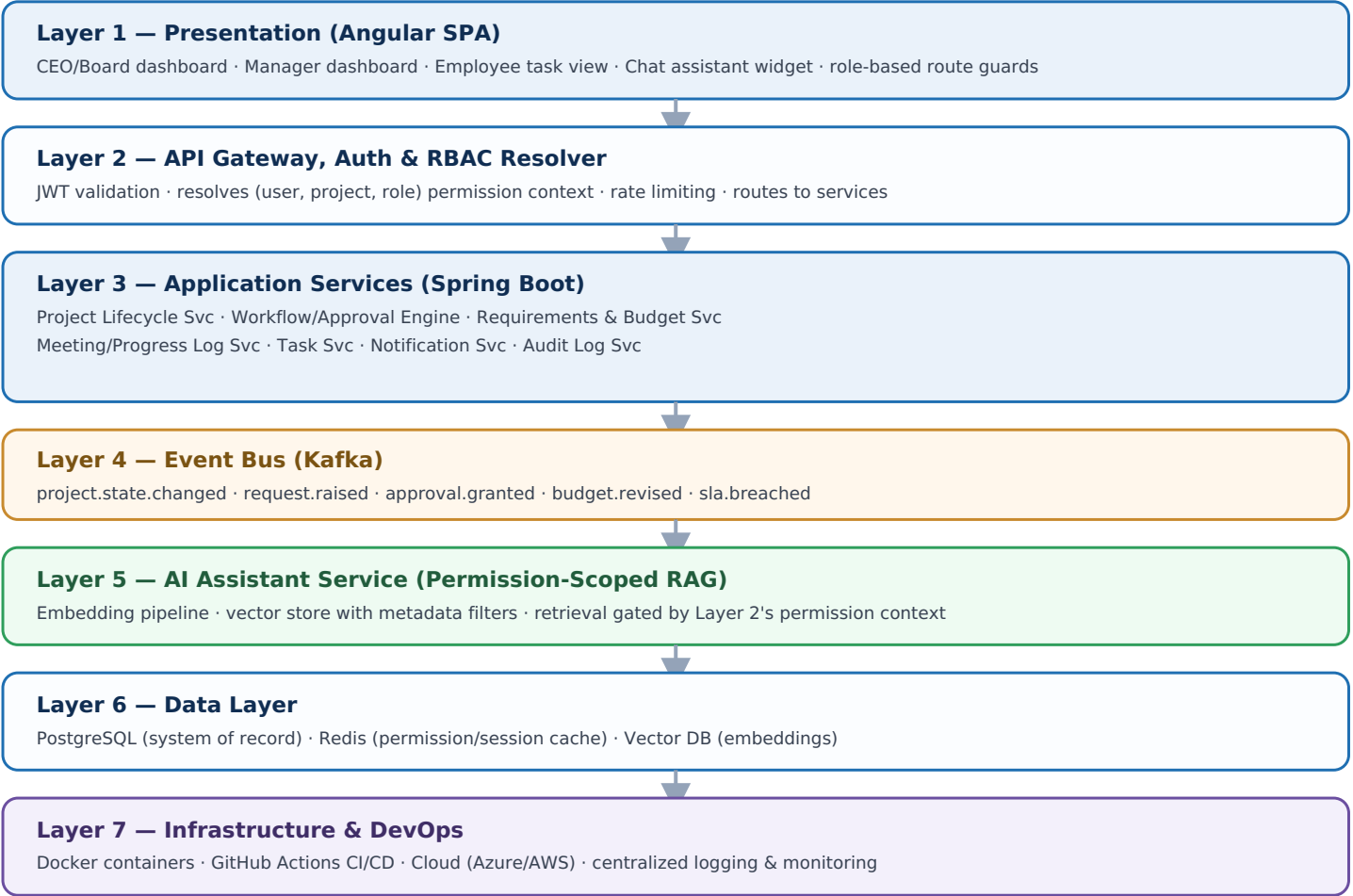
## 5. Full End-to-End Workflow — Part B: Execution to Closure

Continues from the approval decision on the previous page.



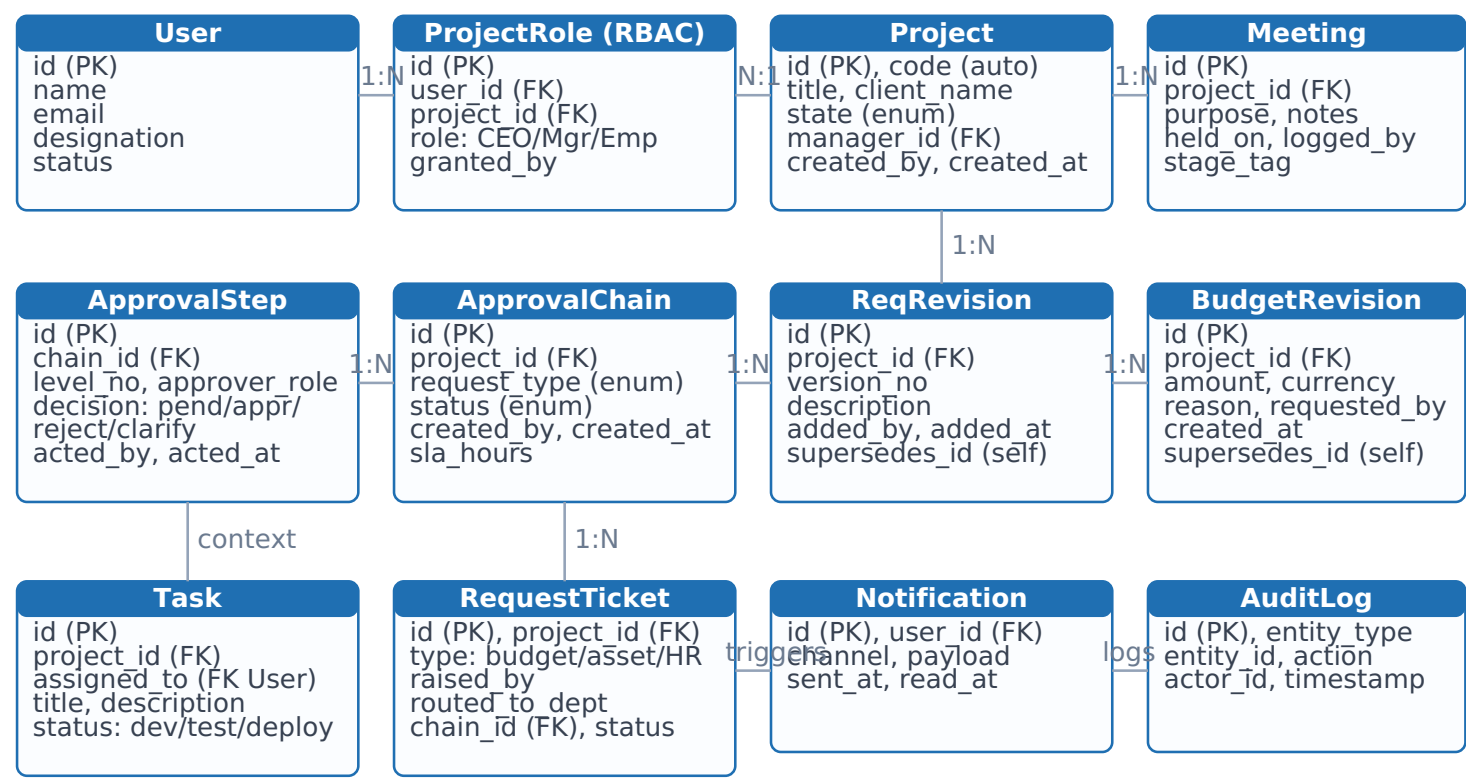


## 6. High-Level System Architecture



Synchronous calls (REST) flow top-to-bottom through Layers 1–3; asynchronous propagation (Kafka) fans out from Layer 4 to Layers 5–6.

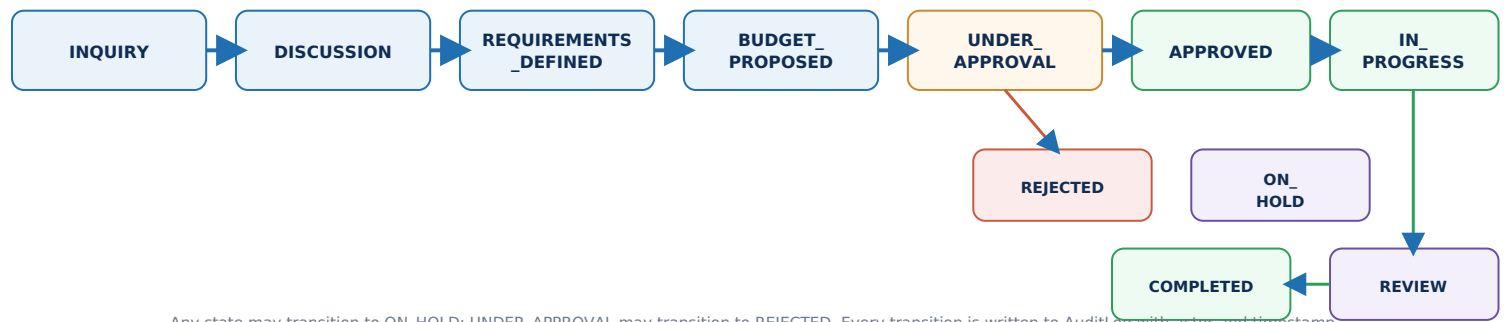
## 7. Entity-Relationship Diagram (Core Schema)



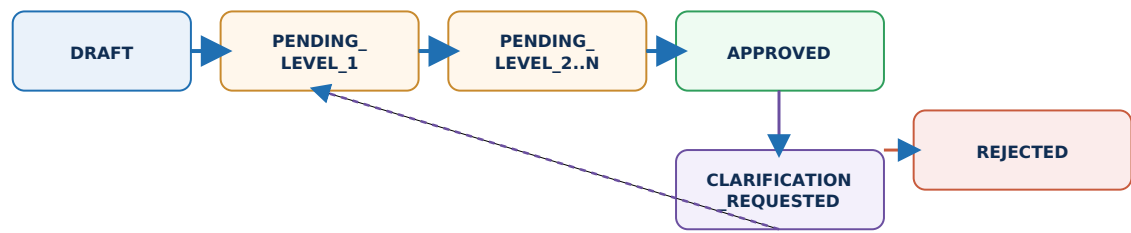
Requirement and Budget entities are append-only revision chains (self-referencing `supersedes_id`) rather than mutable rows, preserving full history. ApprovalChain + ApprovalStep is the generic engine reused by RequestTicket for every request type.

## 8. State Machine Designs

### 8.1 Project Lifecycle State Machine

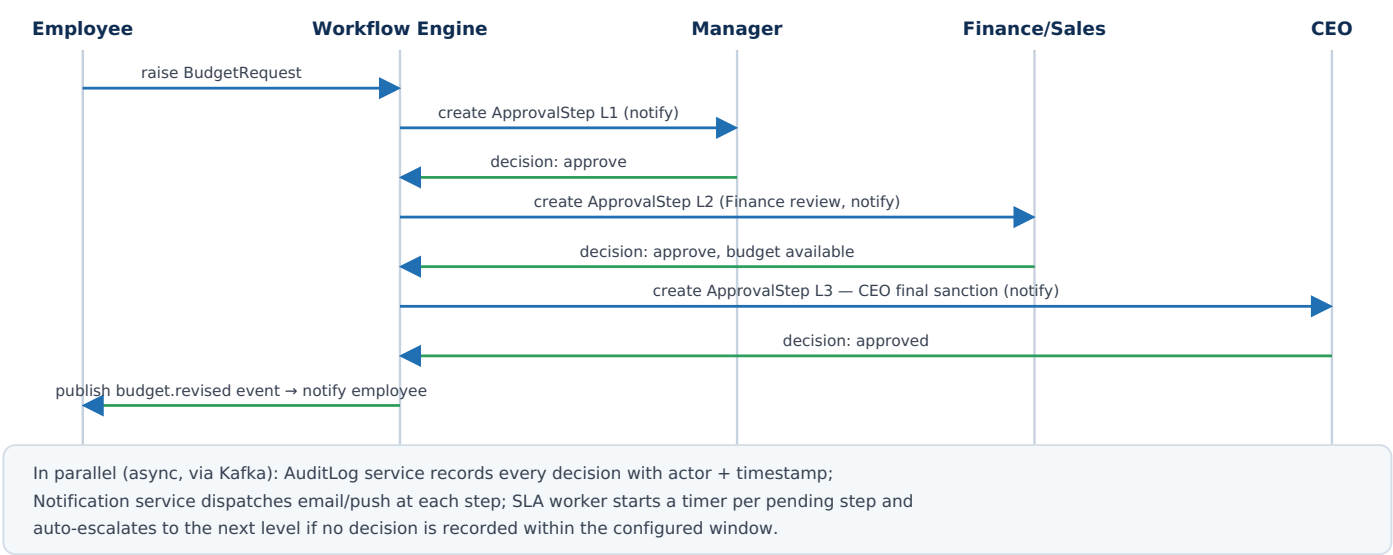


### 8.2 Generic Approval Chain State Machine (reused for Budget / Asset / Clarification requests)



CLARIFICATION\_REQUESTED loops back to the raising level (dashed) instead of outright rejection. SLA-timeout on any PENDING\_\* auto-escalates to the next level.

## 9. Key Flow — Budget Request Sequence Diagram



## 10. Role-Based Access Model

| Role                                 | Visibility Scope                                                                                | Cannot See                                                                               |
|--------------------------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| CEO / Board                          | All projects, all budget/meeting history, all approval chains, org-wide analytics               | —                                                                                        |
| Manager                              | Only projects explicitly assigned to them; requirements, tasks, and progress for those projects | Other managers' projects; board-level discussion notes; org-wide budget totals           |
| Employee                             | Only their own assigned tasks and the requirements relevant to those tasks                      | Project budget, other employees' tasks, meeting/approval history, any unassigned project |
| Finance/Sales/HR (Transition depts.) | Only the specific request ticket routed to them, not the full project                           | Full project context, other departments' tickets                                         |

Enforced at two points: (1) API Gateway resolves and attaches the permission context to every request; (2) the AI Assistant's retrieval layer applies the same filter before returning any content to the LLM.

## 10. AI Assistant & Chatbot Architecture

The assistant is a role- and project-scoped Retrieval-Augmented Generation (RAG) system — not a general-purpose chatbot with the whole database exposed to it. The core principle: **retrieval happens inside the user's permission boundary, before the AI model ever sees the data.**



### Ingestion pipeline (runs continuously via Kafka consumers)

Meeting notes, requirement revisions, task updates, and approval decisions are chunked and embedded as they are created — each embedding is tagged with metadata: project\_id, allowed\_roles, entity\_type, created\_at.

### Guardrails

1) Retrieval filter is enforced at the vector-store query level, not via prompt instructions. 2) The AI model is never given a tool that can bypass the filter (e.g. raw SQL access). 3) Every AI answer logs which chunks were retrieved, for audit purposes.

## Which AI Powers the Chatbot

- **Recommended default:** a hosted large language model accessed via API (e.g. Claude or GPT-class model) for the generation step — best answer quality and reasoning for status summarization, with usage-based cost.
- **Zero-cost alternative for prototyping:** a self-hosted open model (Llama 3 or Mistral) served locally through Ollama — no per-token cost, runs on internal infrastructure, slightly lower reasoning quality.
- **Recommended path:** build and validate the RBAC-scoped retrieval architecture against the free self-hosted model first; swap in a hosted API only if answer quality needs to improve for production rollout — the retrieval/guardrail layer does not change either way.
- Orchestration (prompt assembly, retrieval calls, chat memory) is handled by LangChain, which is free and open-source regardless of which model sits behind it.

## 11. AI / RAG Component Choices — Free / Open-Source First

| Component                                | Free / Open-Source Option                                                                                               | Paid Alternative                                                |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Orchestration framework                  | LangChain or LlamaIndex (both open-source, free)                                                                        | Managed orchestration platforms (vendor RAG-as-a-service)       |
| Embedding model                          | Sentence-Transformers (e.g. all-MiniLM-L6-v2) or BAAI BGE embeddings — run locally via HuggingFace, no API cost         | OpenAI / Cohere embedding APIs                                  |
| Vector database                          | FAISS (Meta, local, free) or Chroma (open-source, self-hosted, free) or Qdrant (open-source, self-hosted, free)         | Pinecone, Weaviate Cloud                                        |
| AI model for generation                  | Self-hosted via Ollama running Llama 3, Mistral, or Phi-3 (fully free, runs on own infra); or free tiers of hosted APIs | Claude API / GPT API (paid per token, higher quality/reasoning) |
| Reranking (optional, improves relevance) | Open-source cross-encoder rerankers from HuggingFace                                                                    | Cohere Rerank API                                               |

Recommended starting stack for a zero-cost prototype: LangChain + Sentence-Transformers + Chroma (all local) + Ollama running a small open model. This validates the RBAC-scoped retrieval architecture before any spend on hosted AI APIs.

### How the Chatbot Is Used, by Role

| Role        | Example query                                   | What the AI is allowed to retrieve                                       |
|-------------|-------------------------------------------------|--------------------------------------------------------------------------|
| CEO / Board | "What's blocking Project PRJ-2026-013?"         | Meetings, requirements, budget, and approval history across all projects |
| Manager     | "Summarize this week's progress on my projects" | Only projects explicitly assigned to that manager                        |
| Employee    | "Is my module tested yet?"                      | Only that employee's own assigned tasks — never budget or other tasks    |

## 12. Technology Stack (Full)

| Layer                | Technology                                                              | Notes                                                 |
|----------------------|-------------------------------------------------------------------------|-------------------------------------------------------|
| Frontend framework   | Angular 17+ (standalone components)                                     | Latest stable major version; no NgModules needed      |
| Frontend language    | TypeScript 5.x                                                          | Strict mode enabled                                   |
| Frontend styling     | SCSS with CSS custom-property design tokens                             | See Section 13 — Frontend Design System               |
| Backend framework    | Spring Boot 3.x                                                         | Requires Java 17+; recommend Java 21 LTS              |
| Backend language     | Java 21 (LTS)                                                           | Latest long-term-support release                      |
| API style            | REST (OpenAPI 3 documented)                                             | Synchronous service-to-service and client calls       |
| Event streaming      | Apache Kafka 3.x                                                        | State-change propagation, async fan-out               |
| Relational database  | PostgreSQL 16                                                           | System of record; append-only revision tables         |
| Cache                | Redis 7.x                                                               | Permission-context and session caching                |
| Vector database (AI) | Chroma / Qdrant (free) or Pinecone (paid)                               | See Section 11 for the full comparison                |
| AI orchestration     | LangChain                                                               | Retrieval, prompt assembly, chat memory               |
| AI model             | Claude / GPT API (hosted) or Llama 3 / Mistral via Ollama (self-hosted) | See Section 10 for the recommended path               |
| Containerization     | Docker (latest stable)                                                  | One image per service — see Section 14                |
| Orchestration (prod) | Kubernetes (optional at scale) or managed container service             | Not required for MVP; see Section 16                  |
| CI/CD                | GitHub Actions                                                          | Test → build → containerize → deploy — see Section 15 |
| Cloud hosting        | Azure App Service / AKS, or equivalent (AWS/GCP)                        | Vendor-agnostic architecture                          |

All versions reflect current stable/LTS releases at time of writing; pin exact versions in each service's build file rather than trailing "latest" tags for reproducible builds.

### 13. Frontend Design System

The frontend has already been implemented as a working Angular build (standalone components, fully responsive). Its design system:

- **Palette:** deep navy ink `#10192E` for the sidebar/header, soft canvas `#F5F7FB` for the page background, white cards, and a confident royal-blue accent `#3457D5` for primary actions.
- **State colors:** distinct colors per lifecycle stage — blue for early stage, amber for under-approval, green for approved/complete, red for rejected, purple for on-hold/clarification — reused consistently across every screen.
- **Typography:** Fraunces (serif) for the wordmark and page titles, giving institutional/governance gravity; Inter (sans) for all functional UI — tables, forms, navigation.
- **Signature element — the Lifecycle Rail:** a segmented progress bar reflecting the project's exact position in its state machine. It appears on dashboard cards, table rows, and the project detail header, so the interface itself teaches the workflow instead of hiding it behind a plain status label.
- **Layout:** fixed left sidebar with role-aware navigation, sticky top bar with search and role/profile switcher, card-based content area.
- **Responsive behavior:** sidebar collapses to a slide-in drawer with backdrop below 1024px; data tables convert to stacked cards below 860px; stat-card grids reflow from four columns to one.
- **Accessibility:** visible keyboard focus rings on all interactive elements; `prefers-reduced-motion` respected throughout.

#### Screens Delivered in the Working Build

| Screen         | What It Shows                                                                                      |
|----------------|----------------------------------------------------------------------------------------------------|
| Login          | Minimal, enterprise-styled sign-in                                                                 |
| Dashboard      | Role-aware stat cards, "projects in motion" feed, recent activity panel                            |
| Project List   | Filterable by lifecycle state; table on desktop, cards on mobile                                   |
| Project Detail | Tabs — Overview, Progress, Requirements & Budget (RBAC-restricted for Employees), Tasks, Approvals |

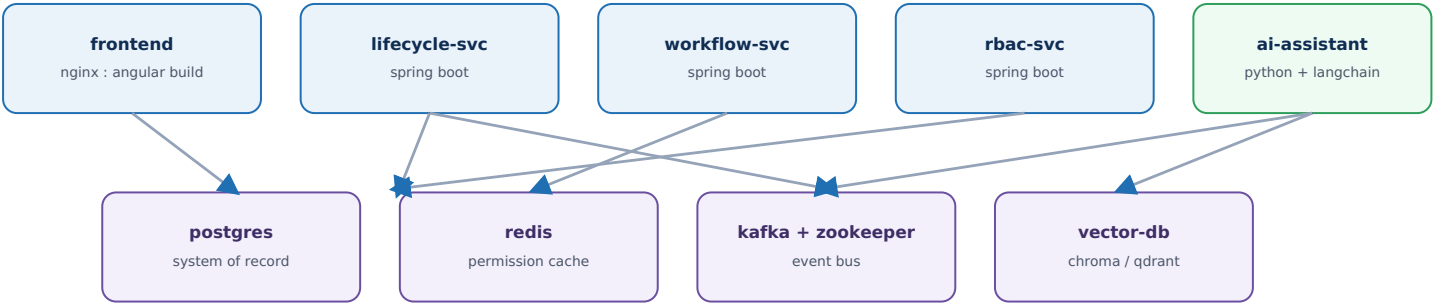
Component structure, mock data service, and full SCSS are packaged as a ready-to-run Angular project — see the accompanying source delivery.



# 14. Docker & Containerization Architecture

- Every backend service (Project Lifecycle, Workflow Engine, RBAC/Auth, Notification, AI Assistant) gets its own `Dockerfile` and its own image — independently deployable and independently scalable.
- The Angular frontend is built to static files and served from a lightweight Nginx image.
- Local development uses a single `docker-compose.yml` that brings up every service plus PostgreSQL, Redis, Kafka, and the vector database together — one command, full stack running locally.
- Each Dockerfile uses a multi-stage build: a build stage with the full SDK/toolchain, and a slim runtime stage (e.g. `eclipse-temurin:21-jre-alpine` for Java services) — keeps production images small.
- Images are tagged with the Git commit SHA (not just `latest` ) and pushed to a container registry as part of the CI/CD pipeline — see Section 15.
- Environment-specific configuration (DB URLs, API keys, feature flags) is injected via environment variables at container start, never baked into the image.

docker-compose.yml — Local Development Stack



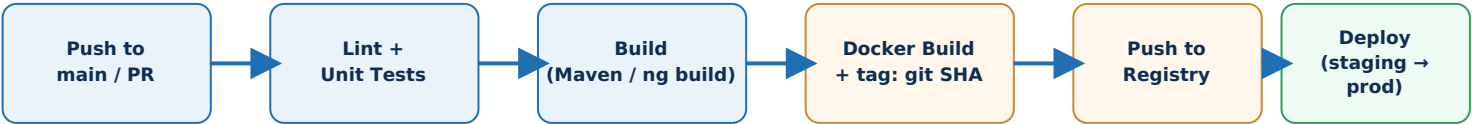
**Multi-stage Dockerfile pattern (applies to every Java service)**

Stage 1 — build: `FROM eclipse-temurin:21-jdk` · runs Maven/Gradle build · produces the jar

Stage 2 — runtime: `FROM eclipse-temurin:21-jre-alpine` · copies only the jar · non-root user · exposes one port

Result: a production image with no build tools, no source code, and a minimal attack surface.

# 15. CI/CD Pipeline



Production deploys require the staging environment to pass smoke tests first; database migrations run as a separate, reviewed step before the new image is promoted — never bundled silently into the deploy step.

## Pipeline Stages

- **Lint + unit tests:** run on every push and pull request; a failing test blocks the merge.
- **Build:** backend services build with Maven/Gradle; the Angular frontend builds with `ng build --configuration production`.
- **Docker build & tag:** each service's image is built and tagged with the Git commit SHA — never overwriting a previous tag.
- **Push to registry:** images are pushed to a container registry (GitHub Container Registry, Azure Container Registry, or equivalent).
- **Deploy:** staging first, automated smoke tests, then manual or automated promotion to production.

## 16. Deployment Architecture

---

- **MVP / early stage:** all backend services deployed as containers on a single managed App Service tier (e.g. Azure App Service for Containers) — simplest operational model, lowest cost, sufficient for initial rollout.
- **At scale:** migrate to a managed Kubernetes service (e.g. AKS) once traffic or service count justifies the added operational complexity — the containerized services do not need to change, only the orchestration layer.
- **Database:** managed PostgreSQL instance with automated backups and point-in-time recovery.
- **Event bus:** managed Kafka (e.g. Confluent Cloud) or self-hosted Kafka cluster, depending on operational preference.
- **Static frontend:** Angular build served via CDN-backed static hosting or the Nginx container, fronted by the same reverse proxy as the API.
- **Observability:** centralized logging and metrics from day one (e.g. Azure Monitor / Grafana + Prometheus) — every AuditLog write and every AI retrieval is queryable in the same place as infrastructure metrics.
- **Secrets:** managed secret store (Azure Key Vault or equivalent) — never committed to source control, never baked into images.

The architecture is intentionally cloud-vendor-agnostic — every component listed has a direct equivalent on AWS or GCP.

## 17. Development Phases — Overview

Given the scope of the platform, development is split into bounded phases rather than attempted as one monolithic build. Each phase ends with a working, demonstrable increment — not a half-finished slice of everything.

### Why Phased

- A working system exists early (by the end of Phase 1) instead of nothing being demonstrable until the very end.
- Each phase has a clear exit criterion — reduces the risk of scope creep inside any one phase.
- The frontend track runs in parallel to backend phases from Phase 1 onward, so UI and API are never blocking each other for long.
- Total estimated time to a launch-ready MVP: **10 weeks** (~2.5 months) for a small, focused team.

### Phase Summary

| Phase | Name                        | Duration            | Primary Deliverable                                                    |
|-------|-----------------------------|---------------------|------------------------------------------------------------------------|
| 0     | Discovery & System Design   | Week 1              | Finalized schema, API contracts, RBAC model                            |
| 1     | Core Lifecycle Engine       | Week 2-3            | Project entity, state machine, meetings, versioned requirements/budget |
| 2     | RBAC & Auth Layer           | Week 4              | JWT auth, project-scoped permission resolver                           |
| 3     | Workflow / Approval Engine  | Week 5-6            | Generic ApprovalChain engine powering budget/asset/clarification flows |
| 4     | Event Layer & Notifications | Week 6-7            | Kafka topics, SLA-escalation worker, email/push dispatch               |
| 5     | AI Assistant (RAG)          | Week 7-8            | Role-scoped retrieval pipeline, chat UI                                |
| 6     | Frontend Build              | Week 3-9 (parallel) | All role-based screens, fully responsive                               |
| 7     | Docker, CI/CD & Deployment  | Week 9              | Containerized services, automated pipeline, staging environment live   |
| 8     | Testing, Hardening & Launch | Week 10             | End-to-end test pass, production launch                                |

Estimates assume a small dedicated team (2-4 engineers). Solo development roughly doubles the timeline; a larger team shortens it but adds coordination overhead.

## 18. Phase 0 & Phase 1 — Setup and Core Lifecycle

### Phase 0 — Discovery & System Design

WEEK 1

- Finalize the ER diagram, state machines, and API contracts (already defined in Sections 7-9 of this document).
- Set up the repository structure, coding standards, and the initial CI skeleton (lint + test running on every push, even before there's much to test).
- Provision base infrastructure: local `docker-compose` stack, empty PostgreSQL schema, empty Kafka topics.
- Design the project-scoped RBAC data model in detail — this is the single most consequential decision in the whole build and is cheapest to get right before any code depends on it.

**Exit criteria:** schema reviewed and frozen for v1; empty services boot and talk to Postgres/Kafka locally via docker-compose.

### Phase 1 — Core Lifecycle Engine

WEEK 2-3

- Build the Project entity and its state machine (INQUIRY → ... → COMPLETED, plus REJECTED/ON\_HOLD branches).
- Build the Meeting log (Progress section) — append-only, chronological.
- Build Requirement and Budget as versioned, append-only revision chains (never in-place edits).
- Build the auto-generated project code sequence and the CEO feasibility-review endpoint.
- Unit + integration tests for every state transition, including the disallowed ones (e.g. cannot jump from INQUIRY straight to COMPLETED).

**Exit criteria:** a project can be created, move through its full lifecycle via API calls, and every transition is visible in AuditLog.

# 19. Phase 2 & Phase 3 — RBAC and Workflow Engine

**Phase 2 — RBAC & Auth Layer**

WEEK 4

- JWT-based authentication; login/token-refresh endpoints.
- Build the (user, project, role) permission resolver as its own service call, cached in Redis for performance.
- Wire every existing Phase-1 endpoint through the RBAC resolver — no endpoint ships without an access check.
- Write explicit negative tests: a Manager on Project A must get a 403 when querying Project B.

**Exit criteria:** all three roles (CEO, Manager, Employee) see correctly scoped data through the API; cross-project access attempts are blocked and logged.

**Phase 3 — Workflow / Approval Engine**

WEEK 5-6

- Build the generic ApprovalChain + ApprovalStep model — configurable levels, per-level approver role, per-chain SLA.
- Implement the three decision outcomes at each level: approve, reject, request clarification (with loop-back to the raiser).
- Wire the Budget Request flow as the first concrete use of the generic engine; Asset and HR ticket types follow using the same engine with different configuration — not new code.
- Build the RequestTicket entity that routes generic requests (budget/asset/HR) to the correct department.

**Exit criteria:** a budget request can be raised, routed through a 3-level chain, and resolved via approve/reject/clarify — end to end, with every step in AuditLog.

## 20. Phase 4 & Phase 5 — Events and AI Assistant

### Phase 4 — Event Layer & Notifications

WEEK 6-7

- Stand up Kafka topics: `project.state.changed` , `request.raised` , `approval.granted/rejected` , `budget.revised` , `sla.breached` .
- Every service from Phases 1-3 becomes a producer on its relevant topic — retrofit, not a rebuild.
- Build the Notification service as a consumer — email/push dispatch on every relevant event.
- Build the SLA-escalation worker — a timer per pending approval step, auto-escalating to the next level (not re-pinging the same approver) on timeout.

**Exit criteria:** a pending approval left untouched past its SLA window auto-escalates and the next approver is notified, with no manual intervention.

### Phase 5 — AI Assistant (RAG)

WEEK 7-8

- Stand up the ingestion pipeline as a Kafka consumer — chunk and embed meeting notes, requirement revisions, and task updates as they're created.
- Tag every embedding with metadata: `project_id` , `allowed_roles` , `entity_type` .
- Build the retrieval endpoint that filters by the caller's permission context *before* the AI model sees anything.
- Start with the free stack (LangChain + Sentence-Transformers + Chroma + Ollama) to validate the scoping logic at zero cost; swap in a hosted AI API only once the guardrails are proven.

**Exit criteria:** three roles ask the same question about the same project and each gets an answer correctly scoped to their own permissions.

## 21. Phase 6 — Frontend Build Track

Runs in parallel starting Week 3, once Phase 1's API contracts are stable enough to build against — does not wait for the backend to be fully finished.

**Phase 6 — Frontend Build**

WEEK 3-9

- Week 3-4:** Angular project scaffold, design token system (palette, type, the Lifecycle Rail signature component), app shell (sidebar + topbar, responsive drawer).
- Week 5:** Login screen; Dashboard with role-aware stat cards and project feed, built against mock data.
- Week 6:** Project List (table/card responsive view) with lifecycle-state filters.
- Week 7:** Project Detail — Overview, Progress, Requirements & Budget (RBAC-restricted), Tasks, Approvals tabs.
- Week 8:** Swap mock data service for real API calls against the now-stable backend from Phases 1-4; wire up the AI chat widget against Phase 5's endpoint.
- Week 9:** Cross-browser and cross-device QA, accessibility pass (focus states, reduced motion), final polish.

**Exit criteria:** every screen renders correctly at desktop and mobile breakpoints, and every role sees only what Section 9's RBAC model says it should.

### Current Status

- The design system, app shell, Login, Dashboard, Project List, and Project Detail screens are already built and verified working (see the accompanying Angular source delivery) — effectively Week 3-7 of this track is complete against mock data.
- Remaining frontend work is wiring to the real backend (Week 8) once Phases 1-4 are live, plus final QA (Week 9).



## 22. Phase 7 & Phase 8 — Deployment and Launch

### Phase 7 — Docker, CI/CD & Deployment

WEEK 9

- Write the production multi-stage Dockerfile for each service (see Section 14).
- Stand up the GitHub Actions pipeline: lint → test → build → containerize → push → deploy (see Section 15).
- Provision the staging environment on the chosen cloud host; wire secrets through the managed secret store.
- Provision managed PostgreSQL, Redis, and Kafka (or self-hosted equivalents) for staging.

**Exit criteria:** a merge to main automatically deploys to staging with zero manual steps.

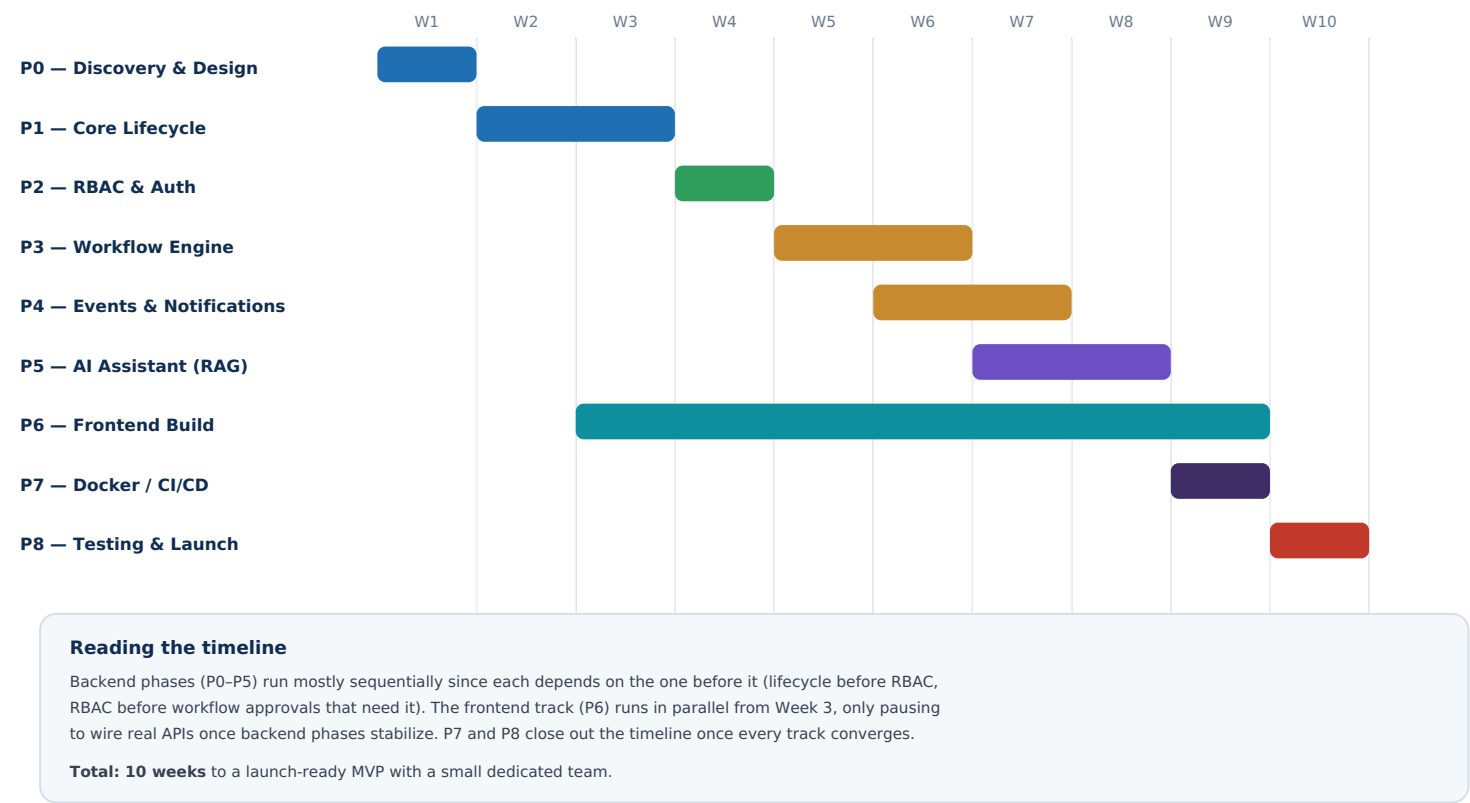
### Phase 8 — Testing, Hardening & Launch

WEEK 10

- End-to-end test pass across all three roles and every workflow path — including reject, clarify, and SLA-escalation branches, not just the happy path.
- Load-test the approval engine and event bus at expected concurrent-project volume.
- Security pass: confirm RBAC boundaries hold under direct API calls (not just through the UI), and that the AI assistant cannot be prompted into leaking out-of-scope data.
- Production environment provisioned; production deploy; monitoring dashboards live from day one.

**Exit criteria:** production is live, monitored, and every role has been walked through their own workflows on the real system.

## 23. Full Timeline (Gantt View)



## 24. Risks & Mitigations

| Risk                                                          | Impact if Unaddressed                                                    | Mitigation                                                                                                            |
|---------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| RBAC modeled as simple global roles instead of project-scoped | Cross-project data leakage; expensive schema rework later                | Freeze the (user, project, role) model in Phase 0 before any dependent code is written                                |
| Budget/requirements stored as mutable fields                  | Loss of history — the platform's core value proposition breaks           | Append-only revision tables from day one (Phase 1), never retrofitted                                                 |
| AI assistant retrieval not scoped at the vector-store level   | An LLM could be prompted into revealing out-of-scope project data        | Enforce the permission filter in the vector query itself, never via prompt instructions alone (Phase 5)               |
| All modules attempted simultaneously                          | Nothing demonstrable for months; scope creep across every module at once | Strict phase boundaries with exit criteria (Sections 18–22)                                                           |
| Notification spam from naive SLA reminders                    | Approvers start ignoring notifications entirely                          | Escalate to the next level on timeout rather than re-pinging the same approver                                        |
| Premature microservice split                                  | Operational overhead without a corresponding scaling benefit             | Modular monolith first; extract only the AI service and Notification service if their load profile genuinely diverges |

## 25. Closing Summary

---

- OPERIVA OS is architected around three ideas that generalize well beyond any single use case: a **generic, configuration-driven approval engine**; **project-scoped RBAC** enforced consistently across the UI and the AI retrieval layer; and an **append-only historical record** for anything that changes over time.
- The same engine that runs a corporate project pipeline can run a law firm's matter-approval process or a government permit workflow — the domain-specific labels change, the underlying engine does not.
- The frontend already exists as a working Angular build with a coherent design system, ready to wire against the backend as each phase lands.
- The 10-week phased roadmap gives a working, demonstrable increment at the end of nearly every phase — not a single all-or-nothing delivery date.
- Recommended immediate next step: freeze the Phase 0 schema and RBAC model, since every later phase builds directly on top of it.